DRAFT FOR ADVISOR REVIEW

STATE, NOT SYNTAX: HARNESS OPTIMIZATION FOR SMALL-MODEL MULTI-TURN TOOL CALLING

A Thesis

Presented to

The Faculty of the Department of Computer Science

California State University, Los Angeles

In Partial Fulfillment

of the Requirements for the Degree

Master of Science

in

Computer Science

By

Sanskar Thapa

December 2026

© 2026

Sanskar Thapa

ALL RIGHTS RESERVED

The thesis of Sanskar Thapa is approved.

Dr. Mohammad Pourhomayoun, Committee Chair

[PLACEHOLDER: Committee member name]

[PLACEHOLDER: Committee member name]

Dr. Chengyu Sun, Department Chair

California State University, Los Angeles

December 2026

ABSTRACT

State, Not Syntax: Harness Optimization for

Small-Model Multi-Turn Tool Calling

By

Sanskar Thapa

Small language models can run on a laptop and call tools for a user, but they often fail when a later call depends on an earlier one. This thesis studies Gemma-4-E2B-it, a model with about two billion effective parameters, on 200 multi-turn tasks from the Berkeley Function Calling Leaderboard. A task is correct only when a simulated backend ends in the right state, such as a car's engine being on.

The best setup leaves the weights fixed. A harness, the program around the model, writes calls as JSON text, records the current state, stops repeated calls, and shows a short card advising the next move. On that harness, writing the same tool list more compactly scored 84 of 200 (42.0%) and 88 of 200 (44.0%) in two paired runs, against 71 and 72 of 200 for the original list. The gain repeated on these tasks. The compact list also cut mean context per request by about 30%. Reduced context is a plausible explanation, but these runs do not isolate the mechanism.

Most failed tasks end with the backend in the wrong state or a required result missing, and the two forced-valid decoding methods we tested made results worse. Without the compact list, the first full-stack run scored 38.5% against 30.5% for JSON text; that gap does not survive correction for multiple comparisons. Fine-tuning did not beat the untuned model.

TABLE OF CONTENTS

Abstract iv

List of Tables viii

List of Figures ix

Glossary x

Chapter

1. Introduction 1

2. Background and Related Work 4

    Benchmark 4

    Output Channel 4

    Constrained Decoding 5

    Harness-Side Control and Test-Time Compute 5

    Training for Tool Use 5

    How This Thesis Differs 5

3. Setup and Methods 7

    Model and Serving 7

    Harness Interventions 7

    Training 9

    Error Labels 9

    Statistics 10

4. Where Small Models Fail 12

    One Call Is Easy; a Chain Is Not 12

    Most Failures Are About the World 12

    The Two Forced-Valid Methods We Tested Did Worse 13

5. What Helps: Harness-Side Control 15
JSON Text over the Native API 15
State Summary 15
Stop on Repeats and the Status Card 15
Full Stack 16
The Best Setup: A Compact Tool List 16
Majority Vote 20
How Sure Are We? 21
6. Fine-Tuning Did Not Help 23
Stop Targets in the First Mixes 23
A Rebuilt Mix Collapsed 24
7. Discussion 26
State, Not Syntax 26
What We Recommend 26
Limitations 27
8. Future Work and Conclusion 29
Future Work 29
Conclusion 29
References 30
Appendix
A. Intervention Details 35
Calls Written as JSON Text 35
The State Summary Block 35
The Stop Rule 35

The Status Card 35

Combining the Full Stack 36

Sampling Five Calls 36

Two-Pass Constrained Decoding 36

The Tool-Name Automaton 36

B. Failure Labels, Exploratory Tests and Training History 37

Planned Comparisons 37

Training History 38

C. Full Scoreboard and Reproducibility 41

Reproducibility 42

Training Recipe 43

Compact Tool List Checks 46

LIST OF TABLES

Table

TABLE 1. COMPACT TOOL LIST AGAINST THE ORIGINAL LIST 19

TABLE 2. THE EIGHT-TEST MCNEMAR FAMILY..... 22

TABLE 3. PRIMARY ERROR LABEL PER FAILED TASK 37

TABLE 4. MCNEMAR TESTS OUTSIDE THE FAMILY 37

TABLE 5. EVERY SCORED CONDITION 41

TABLE 6. RUNS AND LOGS OF THE COMPACT-LIST TEST 43

TABLE 7. FINE-TUNED VARIANTS 45

LIST OF FIGURES

Figure

Fig. 1. The multi-turn tool-calling loop 1

Fig. 2. Primary error labels of failed tasks by condition 13

Fig. 3. The original and compact tool lists 17

Fig. 4. Multi-turn accuracy by condition 20

Fig. 5. Twenty-task gate results 24

GLOSSARY

AST Abstract syntax tree; the single-turn BFCL matcher.
BFCL Berkeley Function Calling Leaderboard.
DPO Direct Preference Optimization.
Full stack JSON text + state summary + stop on repeats + status card.
Greedy decoding Always picking the most likely next token (temperature 0). Repeat runs can still differ.
Harness The program around the model that parses calls, runs tools and records state.
JSON text Calls written as a JSON list in ordinary text, which the harness parses. Logs abbreviate this IFFC.
LoRA Low-rank adapters: small trainable matrices added to frozen weights.
McNemar test A paired test using only tasks where two conditions disagree.
Minimal recipe JSON text + stop on repeats + status card.
Native API Structured `tool_calls` from the server's tool interface. Logs abbreviate this NFC.
Null test A training run that should change nothing if the pipeline is sound.
OODA Observe–orient–decide–act; here only the tag printed on the status card.
pp Percentage points.
Q4_K_M A 4-bit weight-quantization format of llama.cpp.
Self-distillation Training the model on its own answers to simulated tasks that a checker marked correct.
SFT Supervised fine-tuning.
Sign test A paired test counting the tasks on which one condition solved more of its runs than the other; tasks with equal counts are dropped.
Slot One request a server processes concurrently with others.
State summary Harness-written snapshot of facts from executed results.
Status card Four-line card prefixed to the latest user message.
Stop on repeats Ends a turn when the model re-proposes executed calls.
Stop target A training response that is an empty list: no further call.
Tool-name automaton Program that constrains only the tool name.
Turn One user request and everything the model does before handing control back.
Vote of five Majority vote over five sampled calls per step.
Wilson interval A confidence interval for a proportion that stays accurate for small counts.
20-task gate Fixed 20-task screening check (bar 17/20); not a 200-task accuracy.

CHAPTER 1

Introduction

To check a calendar or lock a car, a language model emits a *tool call*: a function name plus arguments. A surrounding program, the *harness*, runs the call against a *backend* (here a simulated vehicle or filesystem) and feeds the result back to the model (Fig. 1). Real tasks chain several dependent calls, which we call *multi-turn* tool calling: look up a value, use it, check the outcome, and know when to stop.

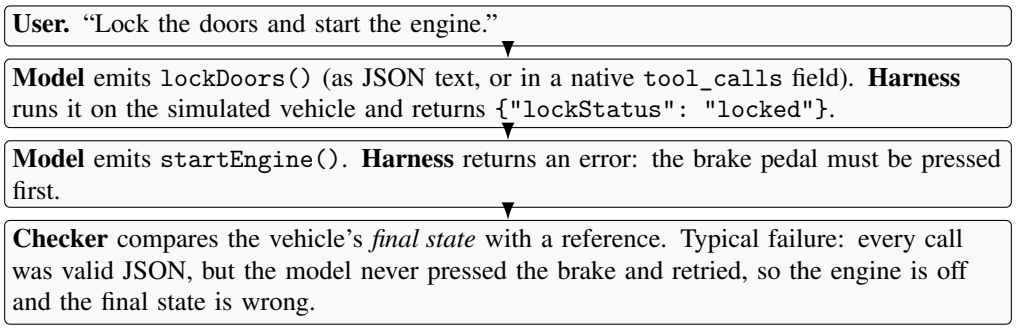


Fig. 1. The multi-turn tool-calling loop studied in this thesis. Grading uses the final state of the simulated backend; whether the calls parse is not enough. Illustrative, not a scored task.

Running this loop locally with a small model keeps private data on the device, works offline and has no per-request cost. The catch is accuracy. We study Gemma-4-E2B-it [1] ("E2B": about two billion effective parameters, meaning the inference footprint; "it": instruction-tuned) on BFCL v4 [2], served with the llama.cpp inference engine [3] on a laptop. Through the server's built-in tool-call field, the native API, it completes 27.0% of 200 multi-turn tasks. The best setup we found, a harness plus a compact tool list, reaches 42.0% and 44.0% (84 and 88 of 200) in two paired runs on the same tasks.

Is the model failing to *write* calls or to *choose* them? The failure labels in Chapter 4 suggest it is mostly the latter: most failed tasks end with a required result

missing or the backend in the wrong state, and forcing valid syntax does not help. We therefore treat the problem as *harness optimization*. The weights stay fixed; we change the loop around the model: how calls are parsed, what state the harness records and shows back, when it ends a turn, and how the next step is framed. We compare harness control with two other families of fixes: forcing syntactically valid output, and fine-tuning the weights. The individual techniques are borrowed or simple. JSON-text calls follow Taheri et al. [4], voting is a standard baseline [5], and the state summary, stop rule, status card, and compact tool list are our own rule-based implementations of familiar ideas (Chapter 2). The contribution is empirical, a controlled comparison on one small model:

1. **A harness plus a compact tool list.** The weights stay fixed. The program around the model records the current state, stops repeated calls, and shows a short card advising the next move. On that harness, writing the same tool list more compactly scored 84 and 88 of 200, against 71 and 72 for the original list, in two paired runs on the same tasks (Chapter 5).

2. **Where the failures are.** Most failed tasks end with a required result missing or the backend in the wrong state. The failure analysis motivated the harness. The two forced-valid decoding methods we tested made results worse (Chapter 4).

3. **Fine-tuning did not help.** The best supervised fine-tune scores below the untuned model, and preference tuning does not beat it. A short fine-tune on the untuned model's own logged answers, which should change little if the training steps are sound, stayed close to the untuned model on a 20-task check. The collapse of the rebuilt training mix therefore points at its data or its much longer training, and these

runs do not separate the two. Two retrains that rebalanced stop and look-up targets also missed the bar (Chapter 6).

The rest of this thesis is organized as follows. Chapters 2 and 3 give the background and the setup. Chapter 4 is the failure analysis. Chapter 5 is the harness, including the compact tool list. Chapter 6 is fine-tuning. Chapters 7 and 8 discuss the results and close. The appendices hold the details.

CHAPTER 2

Background and Related Work

This chapter names the benchmark and the ideas the experiments rely on: how a call is written, what it means to force that writing to be valid, what the program around the model can do, and how training a tool-calling model is usually done.

Benchmark

The Berkeley Function Calling Leaderboard (BFCL) evaluates language models as API callers [2, 6]. Its single-turn categories check one emitted call against a reference with an abstract-syntax-tree (AST) matcher. Its multi-turn categories run the model in an agent loop against simulated backends (for example VehicleControl and GorillaFileSystem, a sandboxed Unix-like filesystem) and compare the backends' final state with a reference. A *turn* is one user request and everything the model does before handing control back; each multi-turn task has several turns. We use the *multi-turn base* split (200 tasks, all tools available). Accuracy is the fraction of the 200 tasks whose final state matches.

Output Channel

With the *native API*, the server receives a tool list and the model returns structured `tool_calls`, as in most production APIs. With *JSON text*, the prompt asks the model to write its calls as a JSON list in ordinary text, which the harness parses. Taheri et al. report that small models follow such JSON instructions more reliably than native tool APIs on single-turn BFCL [4]; we use their single-model variant.

Constrained Decoding

Grammar-constrained generation restricts each next token so that output is always valid, for example to a JSON schema or a list of tool names [7, 8]; Yu et al. constrain tool names with a token-level automaton [9]. Format restrictions can also reduce answer quality [10]. Chapter 4 tests two such methods in a multi-turn loop.

Harness-Side Control and Test-Time Compute

Context-compression methods summarize long histories [11]; our state summary is instead written by the harness from executed results. Observe–orient–decide–act (OODA) controllers structure agent steps [12]; our status card borrows only the name. Guided templates add a reasoning scaffold before calls [13]. Sampling several answers and voting is a standard way to buy accuracy with compute [5]; our vote of five is that baseline.

Training for Tool Use

Specialist small callers such as xLAM [14] and Hammer [15] are trained on synthetic tool data such as ToolACE [16] and APIGen [17]. *Supervised fine-tuning* (SFT) trains the model to reproduce target responses; we use low-rank adapters (LoRA) [18], small trainable matrices added to frozen weights. *Direct Preference Optimization* (DPO) trains on pairs of preferred and rejected responses [19, 20]. Terms used in this thesis are collected in the glossary before Chapter 1.

How This Thesis Differs

Three lines of work are closest. Context compression [11] shortens the history an agent carries; here the state summary is written by fixed rules from executed results, and the compact tool list shortens only the tool descriptions, which repeat on every request.

OODA-style controllers [12] structure each agent step; our status card is one advisory block from fixed rules and adds no model calls. Structured decoding [7, 8] guarantees valid syntax; we test two of our own implementations of that idea on stock `llama-server`, not those engines.

We separate three things. Borrowed: JSON-text calls [4], majority voting [5], the idea of a tool-name automaton [9], and LoRA and DPO training [18, 19]. Our implementation choices: the specific state summary, stop rule, status card, compact tool list, and two-pass constrained decoding described in Chapter 3. The empirical contribution: measuring these harness levers, two forced-valid methods, and fine-tuning against each other on one small model and one benchmark split, with paired tests and repeat runs. We do not claim a new method.

The following chapters use only these ideas. They ask where a small model fails, whether a training-free harness with a compact tool list can help, and whether fine-tuning can.

CHAPTER 3

Setup and Methods

Model and Serving

All results use Gemma-4-E2B-it, quantized with Q4_K_M (a 4-bit weight format of llama.cpp), served by `llama-server` on a laptop. Decoding is greedy (temperature 0) except for majority voting. Greedy runs are not identical from one run to the next (Chapter 5). The machine, the server build, and the flags are in Appendix C.

Two serving settings differ between runs, and neither difference was tested. Most runs used four parallel server slots, and the native API baseline used eight. The earlier runs, including JSON text and the single-lever harness rows, used an older llama.cpp build than the first full-stack run and the compact-list runs. Comparisons across those groups, such as the full stack against JSON text, therefore also cross a server change. Both runs in each compact-list pair used the same build and four slots.

Harness Interventions

All leave the weights untouched. A task is correct only when the multi-turn checker returns valid. Appendix A gives each rule except the compact tool list. Each lever was designed for one problem: the state summary for lost state, the stop rule for repetition, and the status card for whether to act or stop. These are design intents. The error labels in Chapter 5 show what changed, not why.

- **JSON text** instead of the native API. The harness parses the calls.
- **State summary.** After each executed call, the harness writes a short `[STATE]` snapshot of facts from real results (current directory, known files, last calls) into the conversation.

- **Stop on repeats.** The harness ends the turn when the model repeats its previous calls, or when it re-proposes calls already executed in this turn after at least six tool steps. An *ungated* variant drops that minimum.
- **Status card.** A four-line card is prefixed to the latest user message. It advises call, ask, finish, or refuse (the card prints finish as `final`) from fixed rules on the conversation and executed results, and it never sees reference answers. JSON text, stop on repeats, and the card, together, are the *minimal recipe*.
- **Full stack.** JSON text, the state summary, stop on repeats, and the status card, decoded greedily. On its first run this is the harness we compare with JSON text.
- **Compact tool list.** On the full stack, every tool stays in the prompt. The harness states each tool group's shared description once, instead of repeating it, and prints each schema without indentation. It does not remove or retrieve tools. The model, the four pieces above, and the tasks stay the same.
- **Vote of five.** On top of the state summary, at each step the harness samples five candidate calls at temperature 0.7 and executes the most common one. This is the compute-heavy comparison point.
- **Forced-valid decoding.** *Constrained decoding* restricts output in two passes (tool names, then arguments by schema). A *tool-name automaton* constrains only the name, inspired by [9].

A state summary, written from results the harness has already executed, looks like this:

```
[STATE]
cwd:  /workspace/notes
```

```
files:  todo.txt, log.csv
last:  cd(folder='notes')->ok
```

A status card, prefixed to the latest user message, looks like this:

```
[OODA]
goal:  lock the doors and start the engine
slots:  lockStatus=locked
last:  startEngine()->error:  brake pedal must be pressed
next:  call
```

Training

We train LoRA adapters on Gemma-4-E2B-it with MLX [21], merge and quantize to Q4_K_M, and evaluate with JSON text. The *rebuilt mix* has 30,000 rows, about 45% newly simulated episodes plus cleaned public data; earlier mixes use ToolACE and sampled xLAM data. One DPO model is trained too. The *gate* is 20 tasks from the same split, run with the status card; the bar is 17/20. The training settings the repository records are in Appendix

refapp:recipe.

Error Labels

Each failed task gets one primary label, taken from the error message of the BFCL multi-turn checker. The checker goes through the turns in order and stops at the first check that fails. Within a turn it runs three checks in this order, and the label names the first one that fails:

- **Empty turn.** The model's response for a turn that needed calls is empty: no call was executed in that turn.
- **State mismatch.** After the turn, the state of at least one simulated backend differs from the reference state.

- **Under-call.** The backend states match, but the results of the model's calls do not include every result that the reference calls produce, for example because a needed look-up was never made.

Because the checker stops at the first failure, a task with several problems gets the label of its earliest failing turn, and within that turn an empty turn outranks a state mismatch, which outranks an under-call. Every failed task in the labelled runs carries one of these three messages.

Parse failures have no label of their own. With JSON text, an unparseable reply gets one retry. If the retry also fails, the turn ends without that step. If no call had run yet in the turn, the checker then sees an empty turn; if calls had already run, the task can fail as a state mismatch or an under-call instead. On the native API, a reply without a tool call ends the turn, and a tool call whose arguments are not valid JSON is still sent on, marked as an argument error, and fails when executed.

The labels describe outcomes, not causes. A state mismatch says the final state was wrong; it does not by itself show that the model lost track of the state.

Statistics

Each reported accuracy comes with a Wilson 95% interval [22]. It describes uncertainty from the sample of 200 tasks, treating them as drawn from a larger pool of similar tasks. It does not describe how much a rerun on the same tasks would change; that run-to-run variation is reported separately, from repeated runs (Chapter 5).

When two conditions are compared on the same 200 tasks, we use an exact two-sided McNemar test [23]. It counts only the tasks the two conditions decide differently. The p-value is the chance of a split at least that uneven if each such task were

equally likely to go either way. Each test uses one run per condition, so run-to-run flips enter it as if they were real differences, and it says nothing about tasks outside these 200. A p-value above 0.05, or two overlapping intervals, means the data do not separate the conditions. It does not show that they are equal.

Eight comparisons were corrected together with Holm's method [24], so that the chance of any false alarm among them stays at 5%. That family was specified after the harness had been developed on these tasks, and no test in this thesis was preregistered. The compact tool list was a separate test, run twice in opposite order. Which comparisons are in the family, and how the two pairs were combined, is in Appendix B.

CHAPTER 4

Where Small Models Fail

One Call Is Easy; a Chain Is Not

On three single-call BFCL categories (840 cases), Gemma-4-E2B-it through the native API scores 772/840 (91.9%, interval 89.9–93.6%; Table 5). *Simple* offers one function and asks for that call (375/400). *Multiple* offers several functions and asks for the right one (183/200). *Irrelevance* offers functions when the query needs no call (214/240). The two parallel-call categories are excluded because earlier runs on this server hit a tool-grammar bug that truncated multi-call output. On multi-turn tasks, the *same model, quantization and server* completes only 54 of 200 (27.0%). The two settings differ in grading as well as length, but the gap is consistent with carrying a chain of calls through a changing state, rather than choosing one call, being the bottleneck.

Most Failures Are About the World

The loop in Fig. 1 shows how a task fails. The model locks the doors and then tries to start the engine. Both of those calls are valid. The backend says the brake pedal has to be pressed first. The model never presses the brake and retries, so the engine stays off. The vehicle's state then differs from the reference, and the checker labels the task a *state mismatch*. An *under-call* is the other common label: the backend state matches, but a result the reference needs, such as a look-up, is missing.

Fig. 2 breaks down the failed tasks (full counts in Table 3). For every JSON-text condition, state mismatch is the largest bucket (53–109 tasks). With plain JSON text it is followed by empty turns (41) and under-calls (38); the native API has more empty turns (59) and far fewer under-calls (19). With harness control, empty turns fall: the full stack has 20, against 41 for JSON text, while its state mismatches are 68 (60 for JSON text).

Parse failures have no label of their own. They can appear as empty turns or, after earlier calls in the same turn, under another label (Chapter 3). Where format errors were counted, in the compact-list runs, they were rare: at most 1 per run (Appendix C). Those counts are not reported for the other conditions.

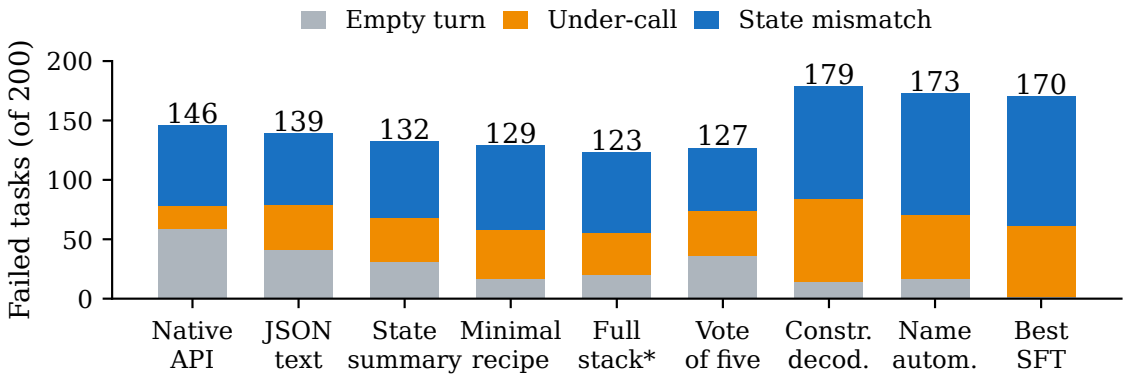


Fig. 2. Why tasks fail. Each bar is one condition on the same 200 tasks (* marks the first full-stack run; Constr. decod. = constrained decoding; Name autom. = tool-name automaton; Best SFT = best fine-tune; conditions as in Chapter 3); its height is the number of failed tasks, split by primary error label (Chapter 3). With harness control, empty turns (grey) fall; wrong final states (blue) remain. Forced-valid output and the best fine-tune shift failures into under-calls (orange) and state mismatches. The compact tool list is omitted.

The Two Forced-Valid Methods We Tested Did Worse

If failures were mostly malformed output, guaranteeing valid output should help. In this setup, the two constrained-decoding implementations we tested performed worse. Two-pass constrained decoding scores 21/200 (10.5%; interval 7.0–15.5%), 20.0 pp below JSON text, with 70 under-calls and 95 state mismatches. The tool-name automaton scores 27/200 (13.5%; 9.4–18.9%), with state mismatch rising to 102. Both are our own implementations on stock `llama-server`; the two-pass program approximates rather than reproduces full grammar engines [8]. The result is about these two implementations in

this loop, not about grammar-constrained decoding in general. A valid call is not necessarily a correct next step.

CHAPTER 5

What Helps: Harness-Side Control

If the bottleneck is choosing the next step, the harness, which sees every executed result, is a natural place to intervene. Every change below is greedy except the vote of five, and none touches the weights. Exact counts and intervals for every condition are in Table 5 (Appendix C).

JSON Text over the Native API

JSON text scores 61/200 against 54/200 on the native API, a lead of 3.5 percentage points that is not statistically significant ($p = 0.32$). The native API run also used eight server slots instead of four (Chapter 3). A seven-step reasoning template on the native API also scored 61/200, the same as JSON text: 19 tasks swap each way, and state mismatches rose to 79. The gap reflects the shape of the prompt, not only the channel that carries the call, so we do not claim the channel switch alone as a gain. Later steps are built on JSON text because the harness can parse and inspect those calls directly. The harness levers were evaluated on JSON text, the stronger baseline.

State Summary

The harness-written snapshot scores 68/200, 3.5 percentage points above JSON text ($p = 0.30$). Empty turns fall from 41 to 31 while state mismatches do not (60 to 64)The labels are consistent with the summary mostly reducing turns with no call; they do not show why.

Stop on Repeats and the Status Card

The gated stop rule scores 65/200 (+2.0 pp, $p = 0.48$). Stopping at the *first* repeat instead scores 54/200, 11 tasks below the gated rule, and all 37 forced stops fell on tasks that failed. Adding the status card gives the minimal recipe, 71/200 (35.5%, +5.0 pp over

JSON text, $p = 0.16$). Relative to stop on repeats alone, empty turns fall from 38 to 17 while state mismatches rise from 60 to 71. The labels are consistent with the card acting mostly on whether the model calls at all, as it was designed to, but they do not isolate that mechanism.

Full Stack

Combining all four greedy changes scores 77/200 (38.5%, interval 32.0–45.4%). Paired against JSON text, it solves 30 tasks that JSON text misses and misses 14 that JSON text solves (Table 2). Against the minimal recipe, the state summary alone, and the vote of five, the paired tests do not separate the full stack from them, which does not show that they are equal. Because the configuration was assembled using earlier results on these tasks, its lead is reported as in-sample (Chapter 7). The next section keeps this harness and changes only how the tool list is written.

The Best Setup: A Compact Tool List

The best setup is the full stack with the tool list written more compactly. It scored 84 of 200 (42.0%) in the first paired run and 88 of 200 (44.0%) in the second. In each pair, the original list is a fresh rerun of the full stack. This is a repeat on the same 200 tasks, not a test on new tasks. The full stack still sends every tool. The list is long because tools that share a group also share a description, and that description was repeated on every tool, with each schema indented. We state each group's shared description once and print the schemas without indentation (Fig. 3). It does not remove or retrieve tools. Nothing else changes. The model, the four pieces of the full stack, and the tasks stay the same. Like the state summary and the card, this is harness-side control delivered through the context.

Original list. Median opening prompt 4,923.5 tokens.

```
[
  {
    "name": "cat",
    "description": "This tool belongs to the Gorilla file system. It is a simple
file system that allows users to perform basic file operations such as navigating
directories, creating files and directories, reading and writing to files, etc.
Tool description: Display the contents of a file of any extension from currrent
directory.",
    "parameters": {
      "type": "object",
      "properties": {
        "file_name": {
          "type": "string",
          "description": "The name of the file from current directory to display.
No path is allowed. "
        }
      },
      "required": [
        "file_name"
      ]
    }
  },
  ...
]
```

Compact list. Median opening prompt 2,888 tokens, a 41% cut.

```
This tool belongs to the Gorilla file system. It is a simple file system that
allows users to perform basic file operations such as navigating directories,
creating files and directories, reading and writing to files, etc.
[{"name": "cat", "description": "Display the contents of a file of any extension
from currrent directory.", "parameters": {"type": "object", "properties":
{"file_name": {"type": "string", "description": "The name of the file from
current directory to display. No path is allowed. "}}, "required":
["file_name"]}}, ...]
```

Fig. 3. File-system tools from the multi-turn tasks, printed both ways. Both lists contain the same tools. The original list repeats the group's shared description on each tool and indents each schema. The compact list states that description once and prints the schemas without indentation. The excerpt is the first tool of the group; an ellipsis marks the rest. The labels are the measured median lengths of the full opening prompt, 4,923.5 and 2,888 tokens, a 41% cut.

How the test was run matters for reading it. Before the first pair, we set the test as a McNemar comparison at the 0.05 level against the first full-stack run. A 10-task check showed that a new run of the original list no longer reproduced that first run's calls, so the first pair used a fresh rerun of the original list instead. After the first pair narrowly missed the 0.05 level, we ran a second pair in the opposite order, with the compact list run first,

as a repeat (Table 1). Both pairs use the same model, the same 200 tasks, and the same other harness parts. The 10-task check, and the token and timing counts, are in Appendix C.

In the first pair the compact list scores 84 of 200 (42.0%, interval 35.4–48.9%) and the original list scores 71 of 200 (35.5%, interval 29.2–42.3%). The compact list solves 13 more tasks: 26 that the original list misses, and it misses 13 that the original list solves (58 both, 103 neither). An exact two-sided McNemar test gives $p = 0.053$, just short of the 0.05 level.

In the second pair the compact list scores 88 of 200 (44.0%, interval 37.3–50.9%), the best single run, and the original list scores 72 of 200 (36.0%, interval 29.7–42.9%). The compact list solves 26 tasks that the original list misses and misses 10 that it solves (62 both, 102 neither), $p = 0.011$.

To count each task once across both pairs, we used a sign test. The compact list does better on 33 tasks and the original list on 17, with 150 ties (exact two-sided sign test, $p = 0.033$). The notes do not record whether this combination was chosen before the second pair ran, so we report it as a follow-up analysis. The gain was 6.5 and 8.0 points and repeated in both pairs on the same tasks. Repeat runs still move individual tasks: between its two runs, the original list changed result on 15 tasks and the compact list on 18.

TABLE 1. COMPACT TOOL LIST AGAINST THE ORIGINAL LIST (SAME MODEL, TASKS, AND OTHER HARNESS PARTS). PAIR 1 COMPARES THE COMPACT LIST WITH A FRESH RUN OF THE ORIGINAL LIST. PAIR 2 RUNS THE COMPACT LIST FIRST. "COMPACT ONLY" COUNTS TASKS THE COMPACT LIST SOLVES AND THE ORIGINAL LIST MISSES. p IS AN EXACT TWO-SIDED MCNEMAR TEST, OUTSIDE TABLE 2. THE SIGN TEST THAT COUNTS EACH TASK ONCE IS IN THE TEXT.

	Compact only	Original only	Both	Neither	p
Pair 1	26	13	58	103	0.053
Pair 2	26	10	62	102	0.011

The compact list reduced mean context per request by about 30% (from about 7,100 to 5,000 tokens) and improved accuracy in both paired comparisons. Reduced context burden is a plausible explanation, but these experiments do not isolate the mechanism. They do not vary context length with the wording held fixed, and the compact list also changes the text the model reads. Its failure labels were not split out. Invalid tool names stay at zero for both lists.

Fig. 4 puts this setup on the same scale as the other conditions. The top two bars are the compact tool list in its two pairs, and the two bars below them are the original list rerun in the same pairs.

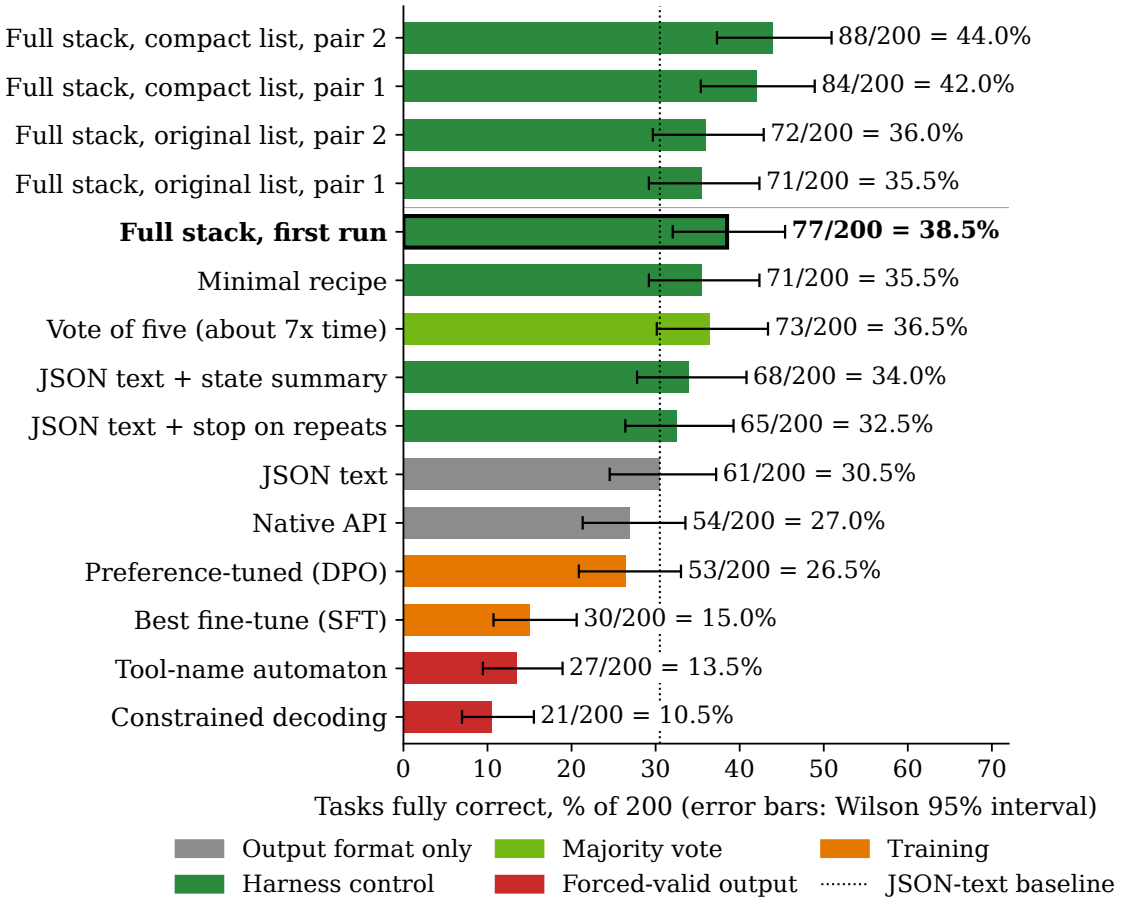


Fig. 4. Multi-turn accuracy on 200 BFCL v4 tasks, Gemma-4-E2B-it. Each bar is a separate run, not a stack of the bars below it. Bars: share of tasks fully correct; error bars: Wilson 95% intervals. Dark green: harness control on JSON text (the first full-stack run is outlined and in bold); light green: majority vote, the compute-heavy comparison; grey: output format alone; red: forced-valid output; orange: training. Dotted line: JSON-text baseline (30.5%). Above the thin line: the four runs of the compact-list test. The top two bars are the full stack with a compact tool list in its two pairs; the two below them are the original list rerun in the same pairs (Table 1).

Majority Vote

The compact list stays greedy. Voting over five samples on top of the state summary scores 73/200 (36.5%), four tasks fewer than the first full-stack run, which solves 24 tasks the vote misses and misses 20 it solves ($p = 0.65$). The vote takes about 7× as long to generate as the greedy full stack, from a single sampling seed. The paired test does not separate the two, which is not the same as showing that they are equal, and

the greedy full stack needs a fraction of the generation time, which matters on a local device.

How Sure Are We?

The compact tool list is the one change we ran twice. Both pairs favored it, and the follow-up sign test that counts each task once gives $p = 0.033$ (Chapter 5). Its failure labels and a direct best-setup versus JSON text comparison are left to the confirmatory runs in Chapter 8. The other harness gaps are less firm, and two separate kinds of uncertainty apply to them.

The first is which tasks happened to be in the sample. The Wilson interval of each harness accuracy extends about six to seven percentage points on either side (Table 5). It treats the 200 tasks as a sample, and it says nothing about what a second run of the same configuration on the same tasks would give.

The second is run-to-run variation, which the intervals do not include. The full stack scored 77 of 200, then 71 of 200 and 72 of 200 on the same tasks. Between its two runs, the original list changed result on 15 tasks and the compact list on 18. A swing of six tasks between runs of one configuration is as large as several gaps between harness rows, so single runs should not be ranked. Against JSON text, the 71-of-200 rerun is ahead by 5.0 percentage points (27 tasks only that run solves, 17 only JSON text, $p = 0.174$).

The McNemar tests in Table 2 use one run per condition, so they share that limit. Of the eight comparisons in the family, two harness gaps fall below 0.05 on their own: the full stack against JSON text ($p = 0.023$) and the minimal recipe against the native API (McNemar $p = 0.033$). Neither stays below 0.05 after correcting for the eight tests. The full stack's first run leads JSON text by 8.0 percentage points, but the order of the harness

rows is not statistically separated. That does not make the rows equal; the data are too few to rank them.

One contrast survives the correction. The best supervised fine-tune solves only 7 tasks that JSON text misses and misses 38 that JSON text solves ($p < 0.001$). The full stack's interval (32.0–45.4%) also lies above those of the best fine-tune (10.7–20.6%) and both forced-valid methods. Preference tuning (26.5%) is not separated from JSON text by these data.

TABLE 2. EXACT TWO-SIDED MCNEMAR TESTS ON PER-TASK RESULTS (200 TASKS EACH, THE SAME TASKS). "A ONLY" AND "B ONLY" COUNT TASKS THAT ONE CONDITION SOLVES AND THE OTHER MISSES. p IS UNCORRECTED; "CORRECTED" MARKS WHETHER IT STAYS BELOW 0.05 AFTER CORRECTING FOR THESE EIGHT TESTS (HOLM'S METHOD). FULL STACK: JSON TEXT + STATE SUMMARY + STOP ON REPEATS + STATUS CARD; MINIMAL RECIPE: JSON TEXT + STOP ON REPEATS + CARD. EXPLORATORY COMPARISONS OUTSIDE THIS FAMILY ARE IN TABLE 4.

A	B	A only	B only	p	Corrected
Full stack (38.5%)	Minimal recipe (35.5%)	19	13	0.38	no
Full stack	Vote of five (36.5%)	24	20	0.65	no
Full stack	State summary (34.0%)	24	15	0.20	no
Full stack	JSON text (30.5%)	30	14	0.023	no
Minimal recipe	JSON text	26	16	0.16	no
Minimal recipe	Native API (27.0%)	37	20	0.033	no
JSON text	Native API	22	15	0.32	no
Best SFT (15.0%)	JSON text	7	38	< 0.001	yes

CHAPTER 6

Fine-Tuning Did Not Help

Fine-tuning is the standard way to adapt a small model to a task. Three terms come up in this chapter. The *rebuilt mix* is a later training set of 30,000 rows that combines newly simulated, execution-checked episodes with cleaned public data. The *gate* is a fixed 20-task check drawn from the same split and run with the status card; the bar is 17 of 20, the untuned model's score with the card. The *null test* is a fine-tune that should change little if the machinery is sound: one pass over the untuned model's own logged answers.

Here no trained model beat the untuned model. The best supervised fine-tune scored 15.0%, significantly below JSON text, and the DPO model scored 26.5%, which the paired test does not separate from JSON text's 30.5%. The runs are evidence about what goes wrong. The version history and the training recipe are in Appendices B and C.

Stop Targets in the First Mixes

The first mixes scored 1/200 (0.5%). A *stop target* is a training response that is an empty list ("no further call"). After a tool result, 78.1% of targets in that data were stop targets, all from ToolACE, while BFCL needs further calls. The low scores are consistent with the model learning to stop after a tool result, but these runs do not isolate that cause. Removing all of those stop targets, and adding recovery examples (an earlier empty reply, then a new request answered with a real call), scored 30/200 (15.0%), our best fine-tune: empty turns nearly vanished (167 to 2), but failures moved to state mismatch (109) and under-call (59).

A Rebuilt Mix Collapsed

The best-validation checkpoint of the rebuilt mix scored 2/20 on the gate (Fig. 5).

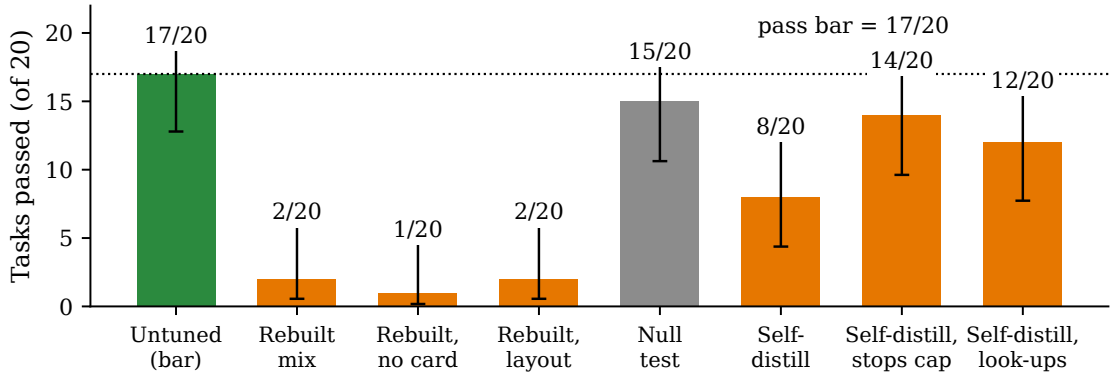


Fig. 5. 20-task gate (the same 20 tasks for every bar; these are not 200-task accuracies). Error bars: Wilson 95% intervals, in tasks. Green: the untuned model with the status card, which sets the bar (17/20). Orange: fine-tuned models, run with the card except the one marked "no card". Grey: the null test. "Layout": continued training on tool lists laid out like the test; "stops cap" and "look-ups": retrains with the stop share capped or look-up episodes added. Every model trained on the rebuilt mix fails (1–2/20), while the null test stays close to the bar. Self-distillation (training on the model's own checker-approved answers to simulated tasks) and its two rebalanced retrains land in between.

Validation loss did not predict the collapse. The collapsed model usually made one call and stopped, even when a setup step was needed (looking up a stock symbol, changing directory, logging in), and invented missing values such as an order number. Without the card it scored 1/20, and continued training on tool lists laid out like the test still scored 2/20. To test the pipeline, the null test trained the untuned model for one pass (39 updates) on 156 of its own logged answers, through the same template, adapter, merge, quantization, and serving steps. It scored 15 of 20, two below the bar of 17 of 20. Their intervals overlap, which does not make them equal, while the rebuilt mix's 2 of 20 is far below the bar. The exact ranges are in Appendix B. The null test suggests that those pipeline steps do not by themselves cause a collapse after a short run of 39 updates. It

does not test them at the rebuilt mix's training length: the evaluated checkpoint came from iteration 22,000 of a run on 30,000 rows, far longer. The rebuilt mix differs from the null test in both content and training length, so these runs cannot say whether its data or its training length caused the collapse.

Self-distillation trains the model on its own answers to simulated tasks, keeping only the turns a checker marked correct. It also missed the bar, at 8 of 20. Its data had shifted toward stop targets and away from look-up calls. Two retrains that each corrected one side of that shift scored 14 and 12 of 20, still below the bar. Rebalancing the data in those two ways did not restore the untuned score, and these runs do not identify what caused the drop. The counts are in Appendix B.

CHAPTER 7

Discussion

State, Not Syntax

Across these conditions, most failed tasks end with the backend in the wrong state or a needed result missing, and the two methods that forced valid syntax did worse. We read this as evidence that the harder part for this model is choosing the next step, including whether to stop, rather than writing a valid call. That reading rests on outcome labels, not on a direct measurement of what the model tracked.

The levers that helped were designed to support that decision: a summary of real results, a gated stop, and a card that advises whether to act. Their measured effect on the labels is mostly on empty turns. The state summary was meant to put lost facts back in view; what falls instead is the number of empty turns. Among the labelled runs, no lever reduces state mismatches, and those remain the largest label. A state mismatch records that the final state was wrong; it does not show by itself that the model lost track of state.

The compact list reduced mean context per request by approximately 30% and improved accuracy in both paired comparisons. Reduced context burden is a plausible explanation, but these experiments do not isolate the mechanism (Chapter 5). The trained models moved failures between labels without reducing state mismatches.

What We Recommend

On this model and these 200 tasks, the best setup we found is the full stack with the tool list written more compactly: JSON-text calls, a harness-written state summary, a gated stop rule, a status card, and the same tools with each group's shared description stated once. All of these are greedy and need no training data or training compute. The compact list added 6.5 and 8.0 points in its two paired runs and cut context per request by

about 30%. The minimal recipe, which leaves out the state summary, is a smaller fallback. Before any fine-tune, screen on task-level checks rather than validation loss. Even the best run, the full stack with the compact list, finishes 88 of 200, and a repeat of the same harness does not pass the same tasks every time. On this benchmark, this model still fails most multi-step tasks.

Limitations

- **One model, one split, one device.** All multi-turn results use Gemma-4-E2B-it on the 200-task base split, on one laptop. The stop threshold, the card's fields, and the training filters were chosen on these same tasks. Other models and harder splits are future work.
- **The harness and the tests were specified on these tasks.** The full stack was put together after earlier results on the same 200 tasks were already known. The family of eight tests was specified after that, and no test was preregistered. The full stack's gain over JSON text does not survive the eight-test correction.
- **Repeatability is not generalization.** Repeated runs of one harness differ on individual tasks. The compact-list gain repeated across two pairs on the same 200 tasks. That shows it was not one lucky run; it does not show that it holds on new tasks.
- **The compact list.** It ran as two pairs outside the eight-test family. The second pair and the sign test that combines them came after the first pair missed the 0.05 level. Its failure labels were not split out, and its mechanism was not isolated. A held-out check is the first confirmatory run in Chapter 8.

- **Serving differed between some runs.** The native API baseline used eight server slots and the other runs four, and the earlier runs used an older server build than the full-stack and compact-list runs. Neither difference was tested for its effect on greedy outputs.
- **The training controls are narrow.** The null test ran for 39 updates; the evaluated rebuilt-mix checkpoint came from iteration 22,000 of a much longer run. The two retrains each changed one share of the data. These runs do not separate training data from training length, and they do not rule out other causes.
- **The screening gate was built from the same split.** The 20 gate tasks were picked using earlier per-task results from the same split, and the bar is the score of the setup the gate is screening. It is designed as a fast screen, with confirmation left to full runs.

CHAPTER 8

Future Work and Conclusion

Future Work

First, *confirmatory runs*: re-score the full stack and the JSON baseline on held-out tasks, the harder BFCL multi-turn splits and a second small model, with matched serving. Second, *retrieval-augmented tool selection*: retrieve the few relevant tools for each step instead of placing every schema in context [6, 25]. The compact list still sends every tool; it is not this selection step. Third, *decision-balanced training data*: vary the look-up and stop shares with everything else fixed.

Conclusion

The best setup is the training-free harness with the same tool list written more compactly. It scored 42.0% (84 of 200) and 44.0% (88 of 200) in two paired runs. In the same pairs, the original list scored 35.5% and 36.0% (71 and 72 of 200). All of these runs used the same 200 tasks, so the gain repeated but has not been tested on new tasks. The compact list reduced mean context per request by approximately 30%. Reduced context burden is a plausible explanation, but these experiments do not isolate the mechanism. Most failed tasks end with the backend in the wrong state or a needed result missing, and the failure analysis motivated the harness. Without the compact list, the first full-stack run scored 38.5% against 30.5% for JSON text. That gain does not survive correction for multiple comparisons. The two forced-valid decoding methods we tested made results worse, and fine-tuning did not beat the untuned model. For a local agent, these harness changes also cost far less than voting or training.

REFERENCES

[1] Gemma Team et al. Gemma 4 technical report, 2026. URL https://arxiv.org/abs/2607.02770.

[2] Shishir G. Patil, Huanzhi Mao, Fanjia Yan, Charlie Cheng-Jie Ji, Vishnu Suresh, Ion Stoica, and Joseph E. Gonzalez. The Berkeley Function Calling Leaderboard (BFCL): From tool use to agentic evaluation of large language models. In *International Conference on Machine Learning*, volume 267 of *Proceedings of Machine Learning Research*, pages 48371–48392. PMLR, 2025.

[3] ggml-org. llama.cpp: LLM inference in C/C++. https://github.com/ggml-org/llama.cpp, 2024.

[4] Yalda Taheri, Mohammad Hassan Heydari, Erfan Naaman, and Afsaneh Fatemi. Small reasoning models are instruction followers in function calling. *arXiv preprint arXiv:2608.22472*, 2026.

[5] Charlie Snell, Jaehoon Lee, Kelvin Xu, and Aviral Kumar. Scaling LLM test-time compute optimally can be more effective than scaling parameters for reasoning. In *International Conference on Learning Representations (ICLR)*, 2025.

[6] Shishir G. Patil, Tianjun Zhang, Xin Wang, and Joseph E. Gonzalez. Gorilla: Large language model connected with massive APIs. In *Advances in Neural Information Processing Systems*, volume 37, pages 126544–126565, 2024. doi: 10.52202/079017-4020.

[7] Brandon T. Willard and Rémi Louf. Efficient guided generation for large language models. *arXiv preprint arXiv:2307.09702*, 2023.

[8] Linzhang Li, Yixin Dong, Guanjie Wang, Ziyi Xu, Alexander Jiang, and Tianqi Chen. XGrammar-2: Dynamic and efficient structured generation engine for agentic LLMs. In *Proceedings of the ACM Conference on AI and Agentic Systems*, CAIS '26, pages 1009–1022, 2026. doi: 10.1145/3786335.3813124.

[9] Minghui Yu, Ke Mu, and Gang Wu. PTC-Decoder: Towards intelligent SLMs on offline resource-constrained edge devices. *arXiv preprint arXiv:2609.30836*, 2026.

[10] Zhi Rui Tam, Cheng-Kuang Wu, Yi-Lin Tsai, Chieh-Yen Lin, Hung-yi Lee, and Yun-Nung Chen. Let me speak freely? A study on the impact of format restrictions on large language model performance. In *Empirical Methods in Natural Language Processing: Industry Track*, pages 1218–1236, 2024. doi: 10.18653/v1/2024.emnlp-industry.91.

[11] Minki Kang, Wei-Ning Chen, Dongge Han, Huseyin A. Inan, Lukas Wutschitz, Yanzhi Chen, Robert Sim, and Saravan Rajmohan. ACON: Optimizing context compression for long-horizon LLM agents. In *International Conference on Machine Learning*, volume 306 of *Proceedings of Machine Learning Research*, pages 55321–55375. PMLR, 2026.

[12] Rongfeng Guo, Yinxuan Huang, Yusen Wu, Maoqing Zhong, Yunlu Chen, Meng Tang, Teng Long, and Vincent Tao Hu. From state to action: OODA-Tool for reliable multi-turn tool use. *arXiv preprint arXiv:2608.24368*, 2026.

[13] Hy Dang, Tianyi Liu, Zhuofeng Wu, Jingfeng Yang, Haoming Jiang, Tao Yang, et al. Improving large language models function calling and interpretability via guided-structured templates. In *Empirical Methods in*

Natural Language Processing, pages 24426–24442, 2025. doi: 10.18653/v1/2025.emnlp-main.1242.

[14] Jianguo Zhang, Tian Lan, Ming Zhu, Zuxin Liu, Thai Hoang, Shirley Kokane, et al. xLAM: A family of large action models to empower AI agent systems. In *Nations of the Americas Chapter of the Association for Computational Linguistics*, pages 11583–11597, 2025. doi: 10.18653/v1/2025.naacl-long.578.

[15] Qiqiang Lin, Muning Wen, Qiuying Peng, Guanyu Nie, Junwei Liao, Xiaoyun Mo, Jiamu Zhou, Cheng Cheng, Yin Zhao, Jun Wang, and Weinan Zhang. Robust function-calling for on-device language model via function masking. In *International Conference on Learning Representations (ICLR)*, pages 85446–85466, 2025.

[16] Weiwen Liu, Xu Huang, Xingshan Zeng, Xinlong Hao, Shuai Yu, Dexun Li, et al. ToolACE: Winning the points of LLM function calling. In *International Conference on Learning Representations (ICLR)*, 2025.

[17] Zuxin Liu, Thai Hoang, Jianguo Zhang, Ming Zhu, Tian Lan, Shirley Kokane, et al. APIGen: Automated PIpeline for generating verifiable and diverse function-calling datasets. In *Advances in Neural Information Processing Systems*, volume 37, pages 54463–54482, 2024. doi: 10.52202/079017-1725. Datasets and Benchmarks Track.

[18] Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. LoRA: Low-rank adaptation of large language models. In *International Conference on Learning*

Representations (ICLR), 2022. URL https://openreview.net/forum?id=nZeVKeeFYf9.

[19] Rafael Rafailov, Archit Sharma, Eric Mitchell, Christopher D. Manning, Stefano Ermon, and Chelsea Finn. Direct preference optimization: Your language model is secretly a reward model. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.

[20] Sunghee Jung, Donghun Lee, Shinbok Lee, Gaeun Seo, Daniel Lee, Byeongil Ko, Junrae Cho, Kihyun Kim, EungGyun Kim, and Myeongcheol Shin. DiaTool-DPO: Multi-turn direct preference optimization for tool-augmented large language models. In *Annual Meeting of the Special Interest Group on Discourse and Dialogue*, pages 397–416, 2025. doi: 10.18653/v1/2025.sigdial-1.32.

[21] Apple Machine Learning Research. MLX: An array framework for Apple silicon. https://github.com/ml-explore/mlx, 2023.

[22] Edwin B. Wilson. Probable inference, the law of succession, and statistical inference. *Journal of the American Statistical Association*, 22(158): 209–212, 1927. doi: 10.1080/01621459.1927.10502953.

[23] Quinn McNemar. Note on the sampling error of the difference between correlated proportions or percentages. *Psychometrika*, 12(2):153–157, 1947. doi: 10.1007/BF02295996.

[24] S. Holm. A simple sequentially rejective multiple test procedure. *Scandinavian Journal of Statistics*, 6(2):65–70, 1979.

[25] Yujia Qin, Shihao Liang, Yining Ye, Kunlun Zhu, Lan Yan, Yaxi Lu, Yankai Lin, Xin Cong, Xiangru Tang, Bill Qian, et al. ToolLLM: Facilitating large

language models to master 16000+ real-world APIs. In *International Conference on Learning Representations (ICLR)*, pages 9695–9717, 2024.

APPENDIX A

Intervention Details

Calls Written as JSON Text

Following Appendix B of [4], the tool list is omitted from the API request; the system prompt asks for a JSON list of `{function_name, parameters}` objects in assistant text. Tool results return as user messages. Calls map to the same BFCL function strings as native calling.

The State Summary Block

After each successful tool execution, the harness writes a compact `[STATE]` block from real results (current directory, known files, last executed calls). The last two turns stay verbatim; older raw outputs are dropped. It is not a model-written summary. An example is in Chapter 3.

The Stop Rule

After a tool result: (1) if the proposed call set equals the previous one, stop; (2) else if every proposed call was already executed in this user request *and* at least six non-empty tool steps have occurred in it, stop; (3) otherwise execute. The six-step window resets on a new user request. The ungated variant applies rule (2) with no minimum.

The Status Card

Prefixed to a working copy of the latest user message (the stored transcript is unchanged), stacked on stop on repeats. Fields: `goal` (latest user question, truncated); `slots` (up to eight key–value facts from executed arguments and results); `last` (last executed call and outcome); `next` (advisory: `refuse` if no tools are available, `ask` if the last result signals a missing or invalid argument, `final` if the user's message is a closing

"thanks/done", otherwise `call`). The hint does not constrain decoding and the card never reads reference trajectories. It is not the multi-pass OODA-Tool of [12]. An example is in Chapter 3.

Combining the Full Stack

The state summary, stop on repeats and status card together: the summary is written into the stored transcript after each executed call, and the card is prefixed to a working copy of the latest user message, as above.

Sampling Five Calls

On the state-summary configuration, each step samples five completions at temperature 0.7, majority-votes the `(function_name, arguments)` tuple, executes the winner once, and continues. It is not a learned verifier.

Two-Pass Constrained Decoding

Two requests per call: the first constrained to an empty list or a list of valid tool names, the second constraining arguments with each tool's JSON schema. No native tool grammar is active. This approximates XGrammar-2's tag-triggered dispatch [8] on stock llama-server.

The Tool-Name Automaton

A live finite automaton restricts the tool name to valid names while arguments are free, on JSON text, not stacked on stop on repeats. Execution accuracy here is not comparable to the F1 reported by Yu et al. [9].

APPENDIX B

Failure Labels, Exploratory Tests and Training History

TABLE 3. PRIMARY ERROR PER FAILED TASK (200 TASKS). NATIVE IS THE NATIVE API. JSON TEXT IS CALLS WRITTEN AS JSON TEXT. STOP = STOP ON REPEATS (GATED); UNGATED = STOP ON FIRST REPEAT; SUMMARY = STATE SUMMARY; MINIMAL = MINIMAL RECIPE; FULL = FULL STACK; VOTE = VOTE OF FIVE; CD = CONSTRAINED DECODING; AUTOM. = TOOL-NAME AUTOMATON; SFT = BEST FINE-TUNE. LABELS AS DEFINED IN CHAPTER 3.

Label	Native	JSON text	Stop	Ungated	Summary	Minimal	Full	Vote	CD	Autom.	SFT
Empty turn	59	41	38	40	31	17	20	36	14	17	2
Under-call	19	38	37	31	37	41	35	38	70	54	59
State mismatch	68	60	60	75	64	71	68	53	95	102	109
Failed	146	139	135	146	132	129	123	127	179	173	170

TABLE 4. EXPLORATORY MCNEMAR TESTS, OUTSIDE THE EIGHT-TEST CORRECTION FAMILY (UNCORRECTED p; COLUMNS AS IN TABLE 2).

A	B	A only	B only	p
State summary (34.0%)	JSON text (30.5%)	20	13	0.30
Vote of five (36.5%)	State summary	17	12	0.46
Stop on repeats (32.5%)	JSON text	11	7	0.48
Minimal recipe (35.5%)	Stop on repeats	26	20	0.46

Planned Comparisons

Eight comparisons were grouped as one family. The family covers the full stack and the minimal recipe against their baselines, the switch from the native API to JSON text, and the best fine-tune. Four of the eight use the full stack. The family was specified after the harness had been developed on these tasks, and it was not preregistered. Four single-lever comparisons were left out of the family (Table 4): the state summary against JSON text, the vote against the state summary, stop on repeats against JSON text, and the

minimal recipe against stop on repeats. The raw p-value is reported, and so is whether Holm's correction [24] changes the conclusion. That correction keeps the chance of any false alarm among the eight tests at 5%.

The compact tool list is not one of those eight tests. Before its first pair ran, the test was set as a McNemar test at the 0.05 level against the first full-stack run. A 10-task check then showed that the original list no longer reproduced that run's calls, so the first pair used a fresh rerun of the original list instead (Appendix C). The first pair missed the 0.05 level ($p = 0.053$). The second pair was run after that, in the opposite order, as a confirmation. To combine the pairs, count how many of its two runs each list solved for each task, and apply an exact two-sided sign test to the tasks where the counts differ. The notes do not record whether this sign test was chosen before the second pair ran, so it is reported as a follow-up analysis. Both pairs use the same 200 tasks as every other comparison.

Training History

LoRA adapters [18] on Gemma-4-E2B-it with mlx-lm; best-validation checkpoint fused and quantized to Q4_K_M; evaluated with JSON text. The rebuilt mix: 30,000 training / 1,500 validation rows; 45/40/10/5% simulated / APIGen / ToolACE / xLAM; 22% empty-list targets (all after a tool result); 41.2% of rows with multi-call targets; no evaluation tool names copied into simulations. DPO: β=0.2, LoRA, scored on JSON text. The training recipe is in Appendix C.

The null test scored 15 of 20 (interval 53–89%); the bar is 17 of 20 (64–95%). The intervals overlap, which does not show that the two are equal. The rebuilt mix scored 2 of 20 (3–30%), far below that bar.

Version History

Dropping only mid-dialogue empty targets from the first mix still scored 1/200; deleting trailing empty answers at serving time scored 4/200. After the repaired mix (30/200), two further filtering variants scored 23/200 and 26/200 (Table 5); each changed several filters at once, so they are not single-factor ablations. The rebuilt mix's training was stopped early at step 26,500 of 32,000; the best validation loss (0.030) was at step 22,000, and that checkpoint is the one evaluated. Continued training for 2,500 steps on tool lists laid out like the test scored 2/20.

Two Targeted Retrains Did Not Restore the Score

Self-distillation (training on the model's own answers to simulated tasks, keeping only turns the task checker marked correct) scored 8/20; in 6 of 12 failures the model said an ID or price was missing and stopped instead of calling the tool that would supply it. Compared with the null-test data, look-up-type calls (a name-matching heuristic) fell from 37.9% to 19.7% of calls and stop targets rose from 23.1% to 33.2%. Two targeted retrains tested this: capping the stop share scored 14/20, and restoring the look-up share (39.6%) scored 12/20. Neither passed. Correcting the shift in the decision mix (stop versus continue, look up versus act) in these two ways was not enough; whether a different rebalancing would work is untested. Leak checks found no evaluation task IDs or tool names in the null-test or self-distillation data.

Self-Distillation Counts

Look-up-type calls: 67/177 in the null-test data versus 78/395 in the self-distillation data; stop targets: 36/156 versus 146/440. Capping the stop share (22.9%) left the look-up share low (20.6%). The look-up-restoring retrain lost 7 tasks the untuned

model solves, and empty turns returned (9 of 66 turns). Look-up counts rest on a name-matching heuristic.

Stop Targets in the Rebuilt Mix

Stop targets were 22% of all rebuilt-mix rows, all after a tool result, a different denominator from the first mix's 78.1% (the share of targets *after a tool result* that were stops). The rebuilt-mix models still under-called, so stop share alone does not appear to explain the collapse.

APPENDIX C

Full Scoreboard and Reproducibility

TABLE 5. EVERY SCORED CONDITION (GEMMA-4-E2B-IT Q4_K_M). THE ID IS THE INTERNAL LOG ID. INTERVAL: WILSON 95%. ROWS OF 20 ARE SCREENING CHECKS, NOT 200-TASK ACCURACIES.

ID	Condition	Result	Acc.	95% interval
Multi-turn base, 200 tasks				
84	Compact tool list on the full stack, second pair (best single run)[a]	88/200	44.0%	37.3–50.9
83	Compact tool list on the full stack, first pair[a]	84/200	42.0%	35.4–48.9
78	Full stack, first run	77/200	38.5%	32.0–45.4
11	Vote of five on the state summary	73/200	36.5%	30.1–43.4
85	Full stack, second rerun, original tool list[a]	72/200	36.0%	29.7–42.9
82	Full stack, fresh rerun, original tool list[a]	71/200	35.5%	29.2–42.3
42	Minimal recipe: card on stop on repeats	71/200	35.5%	29.2–42.3
10	State summary on JSON text	68/200	34.0%	27.8–40.8
30	Stop on repeats, gated at six steps (MIN=6)	65/200	32.5%	26.4–39.3
5	JSON text	61/200	30.5%	24.5–37.2
9	Guided template on the native API	61/200	30.5%	24.5–37.2
4	Native API	54/200	27.0%	21.3–33.5
27	Stop on first repeat (ungated)	54/200	27.0%	21.3–33.5
51	DPO, full train	53/200	26.5%	20.9–33.0
15	Best SFT, repaired mix (SFT-v4)	30/200	15.0%	10.7–20.6
46	Tool-name automaton	27/200	13.5%	9.4–18.9
18	SFT, filters on first-turn targets off[†] (SFT-v6)	26/200	13.0%	9.0–18.4
16	Best SFT + state summary (SFT-v4+STATE)	26/200	13.0%	9.0–18.4
17	SFT, extra first-turn and state filters[†] (SFT-v5)	23/200	11.5%	7.8–16.7
8	Constrained decoding, two-pass (CD)	21/200	10.5%	7.0–15.5
14	SFT first mix, trailing empty answers stripped (SFT-v2 strip)	4/200	2.0%	0.8–5.0
12, 13	SFT first mixes, two variants (SFT-v1, SFT-v2)	1/200 each	0.5%	0.1–2.8
20-task gate (same 20 tasks for every row)				
69	Untuned + card: gate bar	17/20		
70	Rebuilt mix, best-validation checkpoint (SFT-v11, step 22,000)	2/20		
71	Rebuilt mix, no card	1/20		
72	Rebuilt mix, layout continue-train (SFT-v11 fmt2500)	2/20		
73	Null test (v13 null)	15/20		
74	Self-distillation, correct turns only (v13-sd)	8/20		
75	Self-distillation, stop share capped (v13-sdb)	14/20		
76	Self-distillation + simulated look-up episodes (v13-lk)	12/20		
Single-call BFCL v4 categories, native API[‡]				
79	Three categories below (weighted)	772/840	91.9%	89.9–93.6
	Simple (one function offered)	375/400	93.75%	
	Multiple (several functions; pick the right one)	183/200	91.50%	
	Irrelevance (no call needed)	214/240	89.17%	

[†]First-turn filters drop or require training examples where the first response is empty or under-specified; state filters drop continuations inconsistent with executed tool results. [‡]The two parallel-call categories are excluded because earlier runs on this server hit a tool-grammar bug that truncated multi-call output. [a]Run 80 (scores.id 59) is the 10-task original list, 5/10. Run 81 (scores.id 60) is the 10-task compact list, 8/10. The recorded calls differed from the first full-stack run on five tasks, so the 200-task comparison used a fresh rerun. Run 82 is the 200-task original list and run 83 the 200-task compact list (pair 1, $p = 0.053$). Runs 84 (compact list, run first) and 85 (original list) are pair 2 ($p = 0.011$). Across both pairs the compact list does better on 33 tasks and worse on 17 (sign test, $p = 0.033$).

Reproducibility

Hardware: MacBook Pro, Apple M4 Pro, 48 GB, with Metal acceleration (Apple's GPU interface). Temperature is 0 except for voting (0.7, N=5, one seed). The native API baseline used eight server slots; the runs below used four (Table 6). BFCL v4 is read through the `bfcl-eval` package; `pyproject.toml` requires version 2025.8.6.2 or later, there is no lockfile, and the exact version per run was not logged. Failure labels come from the checker messages in the `eval_errors` table of the benchmark database; that database and the run logs below are not in the repository, so the counts here are copied from the paper's tables. Fig. 2 and 5 are regenerated from these counts by `paper/figures/src/make_refocus_figs.py`. Fig. 4 is regenerated from the same scoreboard counts by `thesis/figures/src/make_ladder.py`. The 91.9% interval is the Wilson formula on 772/840 in the single-call log.

TABLE 6. RUNS AND LOGS OF THE FULL-STACK AND COMPACT-LIST TESTS.

Run	What	scores.id	Log
78	Full stack, first run	51	`sft/v13/paper_evals/runs/b_fullstack_mtb200.log`
79	Single-call categories		`sft/v13/paper_evals/runs/a_single_turn_gemma.log`
80	10-task original list, 5/10	59	`R81_slim/smoke_off.log`
81	10-task compact list, 8/10	60	`R81_slim/smoke_on.log`
82	200-task original list	61	`R81_slim/full_off.log`
83	200-task compact list	62	`R81_slim/full_on.log`
84	200-task compact list, second pair	63	`R82_confirm/full_on.log`
85	200-task original list, second pair	64	`R82_confirm/full_off.log`

Earlier runs used llama.cpp build 8111 (commit `11c325c6e`). Runs 78 and 80–85 used commit `d344123fe`. The full stack's harness was run at commit `344ce45` and merged as `83f35ee`. Server flags for those runs: `llama-server --jinja -fa on -ngl 99 -c 65536 --temp 0.0 -np 4 -cb -ub 1024`. Opening-prompt lengths are medians from `R81_slim/*.metrics.jsonl`.

Training Recipe

This section collects the training settings that the repository records, from `sft/lora_config.yaml`, `sft/config.py`, `sft/train_mlx.py`, `sft/README.md`, and the run notes. Settings marked *not recorded* could not be recovered from those files.

Base model. `google/gemma-4-e2b-it` from Hugging Face, converted to full-precision MLX weights with `mlx_lm.convert` before training. The model revision was not pinned. The served base model is the Q4_K_M GGUF of the same model (`bartowski/google_gemma-4-E2B-it-GGUF`).

Data. Rows are chat-format `messages`. The system message is the JSON-text prompt with the tool list, built by the same function the JSON-text evaluation uses

(`benchmark/iffc.py`), so the model sees that prompt in training and at test time. The public sources are `Team-ACE/ToolACE`, `Salesforce/xlam-function-calling-60k` (a 25,000-row sample), and, in the rebuilt mix, `Salesforce/APIGen-MT-5k`; dataset revisions were not pinned. The early mixes pool all of ToolACE with the xLAM sample and split it about 90/5/5 into train, validation, and test with seed 42. Conversations are unrolled so that every assistant turn is a training target.

Template and masking. mlx-lm renders each row with the tokenizer's chat template, and `--mask-prompt` restricts the loss to the assistant response. The exact template and tokenizer files used were not recorded.

Common LoRA settings (`sft/lora_config.yaml`): `mlx_lm.lora`, seed 42; rank 16, scale 32.0, dropout 0.0; adapters on the query and value projections (`self_attn.q_proj`, `self_attn.v_proj`) of 16 layers (`num_layers` 16); learning rate 1.0×10^{-5}; batch size 1 with gradient accumulation 4, so one optimizer update per four rows; maximum sequence length 4,096 tokens; gradient checkpointing on; validation every 200 iterations on 25 batches; a checkpoint every 100 iterations. The optimizer and learning-rate schedule are not set in these files, so they are mlx-lm's defaults for the installed version, which was not recorded. An iteration is one row; with accumulation 4, the 39 updates of the null test correspond to its 156 rows. Later runs overrode some of these values on the command line; for example, an unscored v10 run used a maximum sequence length of 8,192 and seed 53.

Checkpoint selection. For SFT-v1 to SFT-v6 and the rebuilt mix, the checkpoint with the lowest validation loss is evaluated.

Merge, quantize, serve. `mlx_lm.fuse` merges the adapter; llama.cpp's `convert_hf_to_gguf.py` converts the fused model and `llama-quantize` makes the

Q4_K_M file, which `llama-server` serves with the flags above. The llama.cpp build used for each conversion was not recorded.

DPO. One full run (Run 51) with β=0.2 on LoRA, evaluated with JSON text. An earlier smoke run used β=0.2, learning rate 1×10^{-6}, rank 16 and scale 32, for 100 iterations. For the full run, the learning rate, number of updates, and how preferred and rejected responses were paired are not recorded.

TABLE 7. FINE-TUNED VARIANTS, WITH THE SETTINGS THAT DIFFER FROM THE COMMON ONES. AN ITERATION IS ONE ROW; UNDER THE COMMON SETTINGS, FOUR ROWS MAKE ONE UPDATE.

Variant	Data and training	Evaluated checkpoint, result
SFT-v1, SFT-v2	First mixes: ToolACE and the xLAM sample; v2 drops mid-dialogue empty targets	v1 iteration 1,000; v2 iteration 2,000 (val. 0.211); 1/200 each
SFT-v2 strip	SFT-v2 checkpoint; trailing empty answers deleted at serving time	4/200
SFT-v4 (best)	Stop targets after a tool result removed, recovery examples added	Iteration 1,600 (val. 0.196); 30/200
SFT-v5	v4 plus first-turn and state filters	Iteration 1,800 (val. 0.147); 23/200
SFT-v6	v4 plus the state filter only	Iteration 1,800 (val. 0.142); 26/200
Rebuilt mix (SFT-v11)	30,000 train / 1,500 validation rows; 45/40/10/5% simulated / APIGen-MT / ToolACE / xLAM; 32,000 iterations planned, stopped at 26,500	Iteration 22,000 (val. 0.030); 2/20
Layout continue-train	SFT-v11 trained 2,500 more iterations on tool lists laid out like the test	2/20
Null test (v13)	156 of the untuned model's own logged answers, one pass, 39 updates	15/20
Self-distillation (v13-sd)	440 rows of the model's checker-approved answers to simulated tasks	8/20
v13-sdb, v13-lk	Self-distillation with the stop share capped, or with simulated look-up episodes added	14/20; 12/20
DPO	See text	53/200

For the rebuilt mix, the learning rate, gradient accumulation, maximum sequence length, and LoRA settings are not recorded separately, so whether it used the common

settings above is not confirmed. For the v13 runs, the per-run settings and the checkpoint rule are not recorded. Per-run adapter configuration files were not kept in the repository.

Compact Tool List Checks

A 10-task check scored 5 of 10 with the original list and 8 of 10 with the compact list. The recorded calls differed from the first full-stack run on five tasks, so the 200-task comparison used a fresh rerun of the original list rather than that first run (77 of 200).

The context window is 16,384 tokens per request (65,536 shared across four slots). The opening prompt on the original list is about 5,000 tokens. Its median length falls from 4,923.5 tokens to 2,888, a 41% cut (41.3%). Mean context per request falls from 7,107 tokens to 4,990, about 30%. Invalid tool names stay at zero (0 of 1,276 calls with the compact list, 0 of 1,244 with the original list). Invalid arguments are 2 of 1,276 against 9 of 1,244. Format errors are 1 of 1,801 model replies against 0 of 1,773. Mean server time per model request, prompt reading plus generation, with four slots, is 15.4 s with the compact list and 18.8 s with the original list (medians 12.7 s and 15.9 s). Most of the drop is generation (16.2 s to 13.5 s). Prompt reading falls from 2.53 s to 1.86 s.

The second pair shows the same pattern. Mean context per request falls from 7,121 tokens to 5,001, again about 30%, and mean server time per request from 19.4 s to 15.4 s, about 20% faster. Invalid tool names and format errors are zero for both lists. Invalid arguments are 4 of 1,300 calls with the compact list against 5 of 1,255 (0.31% and 0.40%).